Project SENTINEL - Consolidated POD Report

Project SENTINEL aims to develop a domain-specific fraud intelligence system for PayVista Payments Pvt. Ltd. The proposed system, SENTINEL-LM, must analyse transactions, estimate fraud probability, return an ALLOW, REVIEW or BLOCK decision, and provide a plain-language explanation.

The supplied Supabase database kit contains synthetic development data organised into 14 relational tables, one *model_features* view and 43 training-related files in the *sentinel-data* Storage bucket. It supports data exploration, feature validation, model development and testing.

1. Dataset Overview

1.1. Purpose and structure

The dataset supports the development and evaluation of fraud detection, risk scoring, decision-making and explanation generation. Its relational structure separates raw transactions, fraud labels, engineered features, payment outcomes and supporting documents.

Dataset inventory. Column counts are taken from the supplied SQL schema; record counts are reproduced from the existing report and have not been verified against a live database.

Table / view	**Records***	**Columns**	**Purpose**
customers	2,500	6	Customer profiles
merchants	200	5	Merchant identities, categories and locations
devices	10,120	3	Device inventory
transactions	110,000	18	Transaction inputs
payment_outcomes	110,000	5	Payment processing outcomes
labels	110,000	7	Fraud labels and label timing
features	110,000	22	Engineered fraud indicators
legacy_cases	200	9	Historical case examples
documents	5,256	16	Investigation and reference documents
gold_explanations	8,500	8	Explanation examples; review status must be checked
preference_pairs	5,000	9	Preferred and rejected explanation pairs
threshold_sets	1	8	Decision threshold configuration
hard_rules	2	7	Explicit rule-based controls
list_entries	24	6	Configured entity-list entries
model_features (view)	Not verified	33	Joined model-ready view; not a base table

* Report-listed record counts are illustrative inventory figures, not live database measurements. The model_features view is a joined view and its row count is not independently verified.

The 14 tables contain 471,803 table rows in total when each table is counted separately, based on the record counts listed in this report. This is not the number of unique transactions because several tables refer to the same transaction. In particular, transactions, payment_outcomes, labels and features each list 110,000 rows.

The supplied SQL schema defines 33 columns in the model_features view. Column counts for the 14 base tables are listed on page 1 and were counted from the SQL definitions. Actual populated row counts still need to be verified against the live database.

The model_features view combines transaction information, engineered features, customer attributes and labels. The exact number of columns in this view and the individual tables should be verified against the live schema or data dictionary.

1.2. Production versus development data

The client requirements describe a separate production dataset of approximately 12 million transactions across 14 months, January 2025 to February 2026, with a fraud rate of approximately 0.6%. The Supabase kit contains 110,000 synthetic transactions and is intended for classroom development and testing.

2. Data Dictionary

The following dictionary covers the principal transaction fields, engineered features, and supporting database attributes. Examples are illustrative.

2.1. Transaction and customer attributes

Attribute	Data type	Business meaning	Example
transaction_id	TEXT	Unique transaction reference	TX98231
customer_id	TEXT	Pseudonymous customer identifier	C_7f31a9
merchant_id	TEXT	Merchant reference	M_0042
device_id	TEXT, nullable	Device used for payment	D_44be20
amount_paise	BIGINT	Transaction amount in paise	5000000 = ₹50,000
transaction_amount	NUMERIC(14,2)	Transaction amount in rupees	50000.00
payment_channel	TEXT	Payment method or channel	UPI
merchant_category	TEXT	Merchant business category	electronics
transaction_time	TIMESTAMPTZ	Transaction timestamp	2025-11-14T03:12:44Z

customer_age	INTEGER	Customer age in years	27
customer_location	JSONB, nullable	Customer location details	{"city":"Pune"}
merchant_location	JSONB	Merchant location details	{"city":"Delhi"}
account_age	INTEGER	Account age in days	6
previous_transactions	INTEGER	Customer transaction count in the preceding 30 days	14
average_transaction_value	NUMERIC(14,2), nullable	Customer's average transaction value over the preceding 90 days	2350.00
failed_transactions	INTEGER	Failed attempts in the preceding 24 hours	4
split	TEXT	Dataset partition for controlled model development	train
is_synthetic	BOOLEAN	Indicates synthetic rather than real data	true

The customers table also contains created_at (TIMESTAMPTZ, account creation time), home_city (TEXT), region (TEXT) and is_synthetic (BOOLEAN). The merchants table contains synthetic_name (TEXT), merchant_category (TEXT), merchant_location (JSONB) and is_synthetic. The devices table contains device_kind (TEXT) and is_synthetic, in addition to device_id.

2.2. Labels and payment outcomes

Attribute	**Data type**	**Business meaning**	**Example**
fraud_label	SMALLINT	Confirmed target: 1 = fraud, 0 = genuine	1
label_source	TEXT	Source of the fraud verdict	investigator

label_available_at	TIMESTAMPTZ	Time the label became eligible for use	2026-01-15T00:00:00Z
eligible_at_next_split_start	BOOLEAN	Indicates label timing eligibility for a later split	true
outcome_time	TIMESTAMPTZ	Time the payment outcome was recorded	Timestamp
status	TEXT	Payment processing outcome	success
failure_code	TEXT, nullable	Reason for payment failure	insufficient_funds

labels keeps the fraud target separate from the raw transaction input. This is important because a live scoring model must not receive the answer it is expected to predict. Payment success or failure must also not automatically be treated as proof of fraud.

2.3. Engineered features

Attribute	**Data type**	**Business meaning**	**Example**
amount_to_avg_ratio	DOUBLE PRECISION	Current amount divided by average transaction value	21.28
is_new_device	BOOLEAN, nullable	Device not seen for the customer in the previous 90 days	true
customer_merchant_distance_km	DOUBLE PRECISION, nullable	Geographic distance between customer and merchant	1,170.5
is_new_city	BOOLEAN, nullable	Transaction city is new relative to the relevant history	true
hour_of_day	INTEGER	Transaction hour	3

is_night	BOOLEAN	Whether the transaction occurs from midnight to 5 a.m.	true
has_recent_failures	BOOLEAN	Whether at least three recent failures occurred	true
is_new_account	BOOLEAN	Whether account age is below 30 days	true
txn_count_last_1h	INTEGER	Transactions in the preceding hour	5
txn_amount_last_24h_paise	BIGINT	Transaction amount accumulated over the preceding 24 hours, in paise	7500000
distinct_customers_per_device_7d	INTEGER, nullable	Number of distinct customers associated with a device over seven days	4
combined_risk_flags	INTEGER	Count of selected risk flags	4
mcc_target_encoded_rate	DOUBLE PRECISION	Training-derived merchant-category fraud-rate encoding	0.012
mcc_encoding_as_of	TIMESTAMPTZ	Timestamp associated with the category encoding	Timestamp
mcc_encoding_count	INTEGER	Number of observations supporting the encoding	250

missing_customer_location	BOOLEAN	Indicates unavailable customer location	false
missing_device	BOOLEAN	Indicates unavailable device information	false
cold_start	BOOLEAN	Indicates insufficient history for some features	false
reason_codes	JSONB	Structured explanation or risk-reason indicators	["NEW_DEVICE"]
feature_as_of	TIMESTAMPTZ	Time at which feature values were calculated	Timestamp
feature_version	TEXT	Feature logic version	v1.0

The features table also stores transaction_id (TEXT) as its transaction reference. Nullability is important: an unavailable feature should not automatically be interpreted as a low-risk value.

2.4. Supporting tables and attributes

The remaining schema supports traceability, training and operational controls.

Table	**Attributes and meaning**
legacy_cases	legacy_case_id (TEXT), customer_id (TEXT), transaction_time (TIMESTAMPTZ), closed_at (TIMESTAMPTZ), amount_paise (BIGINT), payment_channel (TEXT), fraud_label (SMALLINT), evidence (JSONB), is_synthetic (BOOLEAN). Stores case evidence and outcomes.
documents	document_id (TEXT), kind (TEXT), title (TEXT), transaction_id (TEXT, nullable), legacy_case_id (TEXT, nullable), split (TEXT), published_at (TIMESTAMPTZ), content (TEXT), content_sha256 (TEXT), template_family (TEXT), cpt_eligible (BOOLEAN), review_status (TEXT), storage_path (TEXT), record_in_shard

	(INTEGER), metadata (JSONB), is_synthetic (BOOLEAN). Supports corpus management and provenance.
gold_explanations	example_id (TEXT), transaction_id (TEXT), split (TEXT), prompt (TEXT), target (JSONB), fraud_label (SMALLINT), review_status (TEXT), is_synthetic (BOOLEAN). Stores explanation prompts and targets.
preference_pairs	pair_id (TEXT), transaction_id (TEXT), split (TEXT), prompt (TEXT), chosen (JSONB), rejected (JSONB), rejection_reason (TEXT), review_status (TEXT), is_synthetic (BOOLEAN). Supports preference optimisation.
threshold_sets	threshold_id (TEXT), version (INTEGER), mcc_group (TEXT, nullable), medium_from, high_from, critical_from (NUMERIC(4,3)), status (TEXT), is_synthetic (BOOLEAN). Defines versioned risk thresholds.
hard_rules	rule_id (TEXT), code (TEXT), expression (JSONB), action (TEXT), priority (INTEGER), enabled (BOOLEAN), is_synthetic (BOOLEAN). Stores explicit decision rules.
list_entries	entry_id (TEXT), list_type (TEXT), entity_id (TEXT), effective_from (TIMESTAMPTZ), reason (TEXT), is_synthetic (BOOLEAN). Records time-effective entity-list entries.

3. Business Mapping to Client Requirements

Client requirement	**Dataset support and implementation**
Detect card and UPI fraud	Transaction amount, channel, time, customer history, merchant category, device and location features
Return a fraud probability	fraud_label provides the supervised target; features supplies model inputs
Return ALLOW / REVIEW / BLOCK	Model score combined with validated thresholds and high-priority hard rules
Explain the decision	documents, legacy_cases, gold_explanations, preference_pairs and reason_codes
Train on PayVista data	Use authorised production data and approved corpora inside PayVista's AWS environment; the supplied kit supports development only
Avoid future-data leakage	Respect split, label_available_at, feature_as_of and document eligibility

4. Data Quality Observations

The client's production specification identifies approximately 3% missing customer-location values and some older UPI transactions with missing device IDs. Rare transactions exceed ₹10 lakh, and fraud represents approximately 0.6% of the production dataset—roughly one fraud case per 167 transactions. These are expected production characteristics, not verified findings about the synthetic Supabase data.

Required checks include:

- **Missing values:** quantify nulls in locations, device IDs and average transaction values; verify missingness flags.
- **Duplicates:** check uniqueness of primary keys and duplicate document content.

5. AI/ML Opportunities

1. **Supervised fraud classification:** Train ML model using transaction and engineered features. Handle class imbalance with suitable class weighting or sampling applied only to training data.
2. **Risk scoring and decisioning:** Calibrate probabilities and evaluate precision, recall, PR-AUC, false declines and fraud loss. Choose ALLOW/REVIEW/BLOCK thresholds using business costs and review capacity.
3. **Domain-adapted language model:** Use approved investigator notes, dispute narratives, playbooks and reference documents for continued pretraining where appropriate. Use reviewed prompts and gold explanations for supervised fine-tuning.
4. **Grounded explanations:** Generate concise reasons based on actual features and approved evidence; prohibit invented evidence and sensitive-data reproduction.

6. Discussion Notes: Observations, Questions and Decisions

Key observations

- The dataset separates transactions, labels and features, supporting safer model development.
- Transaction context is important: amount anomalies, new devices, new locations, recent failures and transaction velocity may jointly indicate risk.
- Fraud labels may become available well after authorisation; label timing must be respected.

Questions to resolve

- Which document types are eligible for continued pretraining and which are approved explanation targets?
- How will model drift, threshold changes and human overrides be monitored?

Proposed decisions and learnings

- Treat the supplied Supabase kit as synthetic development data.

- Preserve train, validation and holdout boundaries.
- Validate data quality, explanation faithfulness, calibration, privacy and end-to-end latency before deployment.

7. Individual POD Contributions

POD member / role	Contribution to record
Member 1-Poorva Wadkar	Explained what type of files we have in structure and unstructured dataset, then columns, their data types, table schema and keys.
Member 2 - Chouda Reddy	Explored the supabase schema, transaction fields present, table relationships, keys, data types.
Member 3 - Rajesh S	Explained the business problem
Member 4 - Saurabh Naik	Explored the null values, columns which can be helpful for detecting fraudulent transactions like missing location or average traction amount details.
All POD members	Discussed the nature of data, why the particular transaction might be fraudulent.